VINUNIVERSITY



Ngày 23

# LangGraph, Deep Agents và agent orchestration

Khi nào kỹ sư viết đường đi, khi nào để model tự chọn, và cách ghép cả hai.

GIẢNG VIÊN
[Tên giảng viên]

PHIÊN BẢN
langgraph 1.2 · deepagents 0.7

THỜI LƯỢNG
4 giờ · 7 phần · lab buổi riêng



# Bảy phần, một câu hỏi: ai chọn bước tiếp theo?

VINUNIVERSITY

## Nền tảng

01 Phổ deterministic ↔ agentic
02 LangGraph
03 Durable execution & HITL

## Từ agent đến harness

04 Agent và Deep Agents
05 Bản đồ framework

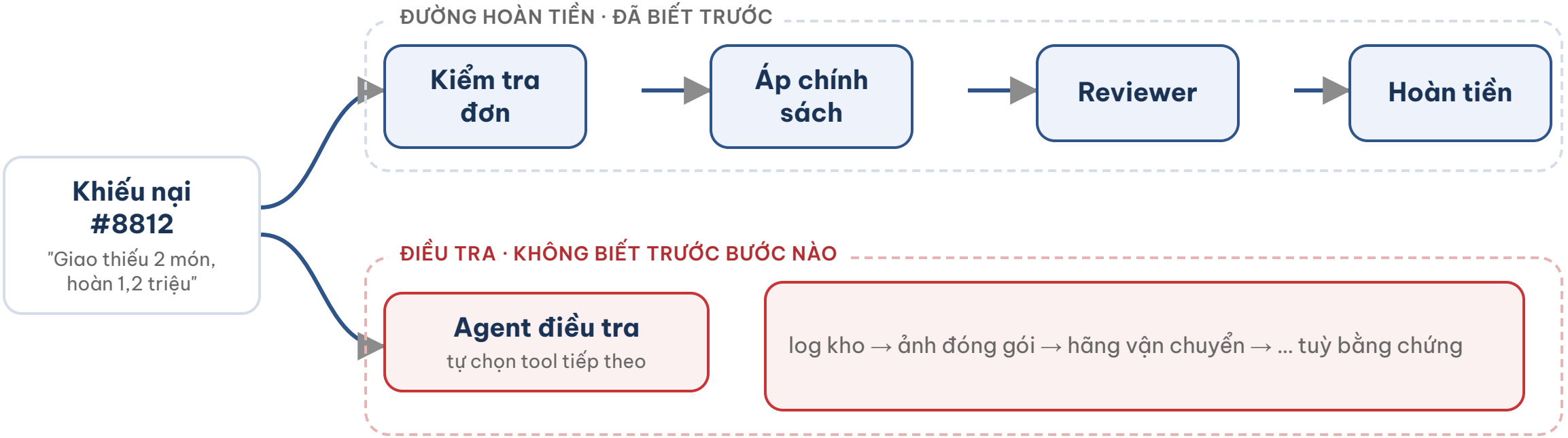
## Toàn cảnh

06 Multi-agent và frontier
07 Hero: hệ thống lai

**ỨNG DỤNG CHẠY SUỐT BUỔI**

Trợ lý **hoàn tiền và điều tra khiếu nại** của một sàn thương mại điện tử.

# Một khiếu nại, hai việc rất khác nhau bên trong




●●●●● **Hệ thống thật cần cả hai**: phần trên do kỹ sư viết, phần dưới để model chọn.

VINUNIVERSITY

01

# Phổ deterministic ↔ agentic

"Agent" là một mức độ, không phải nhãn có/không — và mỗi mức có giá.

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Đặt một hệ thống lên phổ sáu mức.

02 Nhận ra năm mẫu workflow và khi nào cần vòng lặp agent.

03 Chọn điểm trên phổ theo bài toán, không theo trào lưu.

CÂU HỎI MỞ ĐẦU Chatbot có gọi một tool tra đơn hàng: đó đã là agent chưa?

# Sáu mức trên phổ: quyền chọn bước chuyển dần sang model

| Code cứng | Một lần gọi LLM | Chain | Router | State machine | Agent tự chủ |
|---|---|---|---|---|---|
| if/else, không gọi LLM | tóm tắt khiếu nại | trích → phân loại → soạn thư | LLM chọn 1 trong 3 nhánh | LLM routing lặp lại, code giới hạn bước chuyển | model chọn tool và thứ tự |
| KỸ SƯ CHỌN | KỸ SƯ CHỌN | KỸ SƯ CHỌN | MODEL CHỌN TRONG KHUNG | MODEL CHỌN TRONG KHUNG | MODEL CHỌN |

deterministic — agentic

> Model quyết cả tool lẫn thứ tự. Harness vẫn cấp môi trường, quyền và điều kiện dừng.

Nguồn: Phổ do bài giảng điều chỉnh từ Harrison Chase (05/07/2024); ở Chase, mức cuối còn cho hệ tự đổi prompt, tool, code.

VINUNIVERSITY

# Năm mẫu workflow giải phần lớn bài toán, trước khi cần agent

### 1 · Prompt chaining

Bước sau nhận đầu ra bước trước.

### 2 · Routing

Phân loại rồi gửi sang nhánh riêng.

### 3 · Parallelization

Chia phần độc lập, chạy song song.

### 4 · Orchestrator–workers

LLM tự chia việc lúc chạy, giao cho worker.

### 5 · Evaluator–optimizer

Một LLM viết, một LLM chấm, lặp.

### Agent tự chủ

Model chọn tool và khi nào dừng.

Chỉ dùng khi số bước không đoán trước được.

Nguồn: Anthropic, "Building effective agents" (19/12/2024); ví dụ hoàn tiền là của bài giảng.

# Dự đoán: bài toán này hợp với mẫu nào?

Mỗi tối, hệ thống đọc 300 đánh giá sản phẩm mới. Chưa biết trước sẽ có bao nhiêu chủ đề; mỗi chủ đề cần một báo cáo riêng rồi gộp thành một bản tin.

A Prompt chaining

B Parallelization

C Orchestrator–workers

D Agent tự chủ

Bạn chắc đến đâu? Đoán | Khá chắc | Chắc chắn

Làm lại

# Kéo thanh trượt: mỗi bước thêm vào nhân rủi ro, không cộng

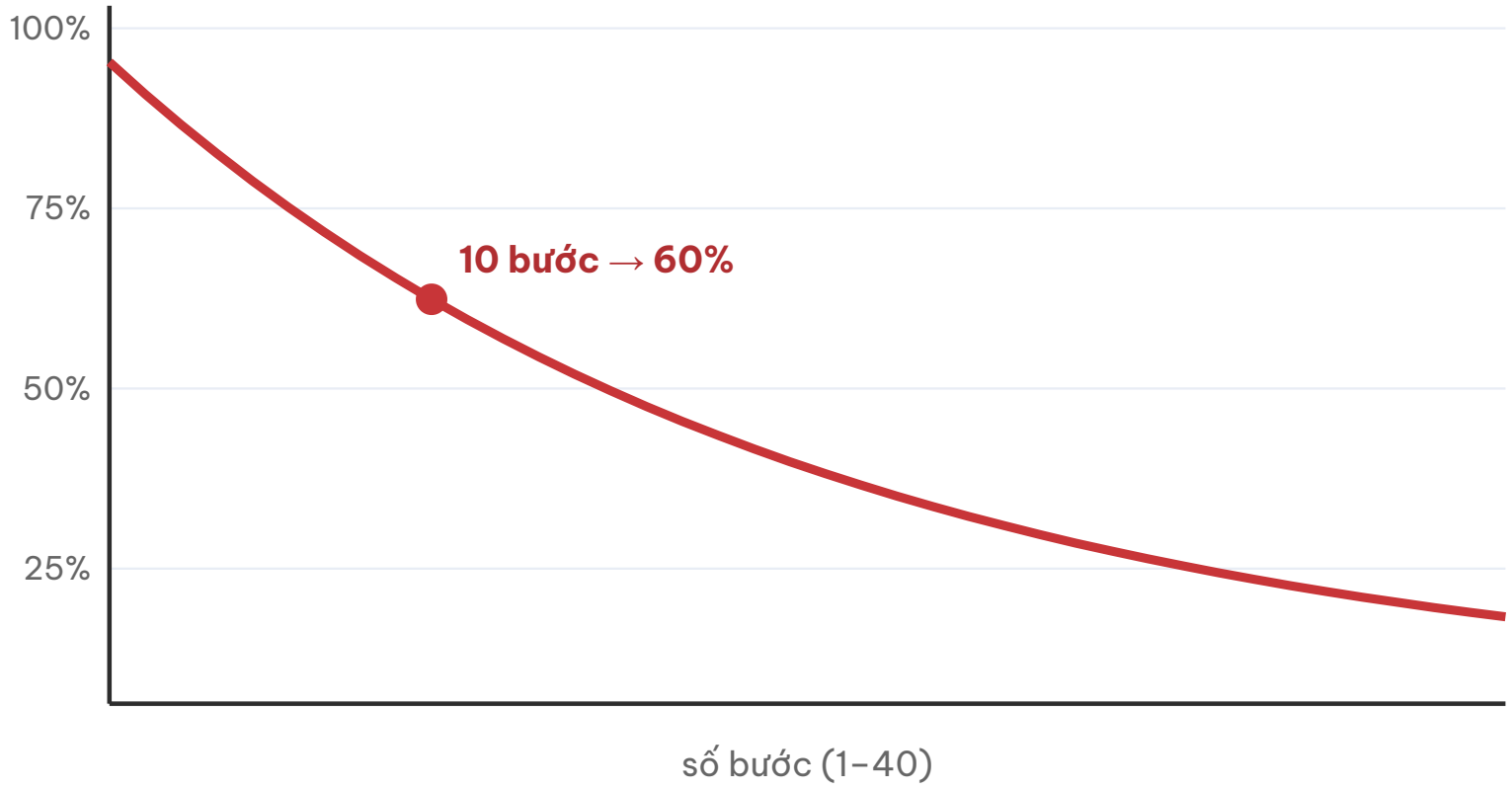
Mỗi bước đúng 95

Số bước 10

**ĐỌC BIỂU ĐỒ**

Xác suất cả chuỗi đúng = $p^n$. Ở 95% mỗi bước, 10 bước còn **60%**.

**Minh hoạ** · mô hình đồ chơi: các bước độc lập, không thử lại, không kiểm tra. Hệ thống thật có retry và cổng kiểm.




Nguồn: Lập luận lỗi cộng dồn: Anthropic, "Building effective agents" (2024). Con số là phép tính minh hoạ, không phải tỉ lệ thành công đo được.

# Tính agentic có giá: token và tiền tăng gấp nhiều lần

**≈4×**

token của một agent so với chat thường

Nguồn: Anthropic, multi-agent research system (13/06/2025)

**≈15×**

token của hệ multi-agent so với chat thường

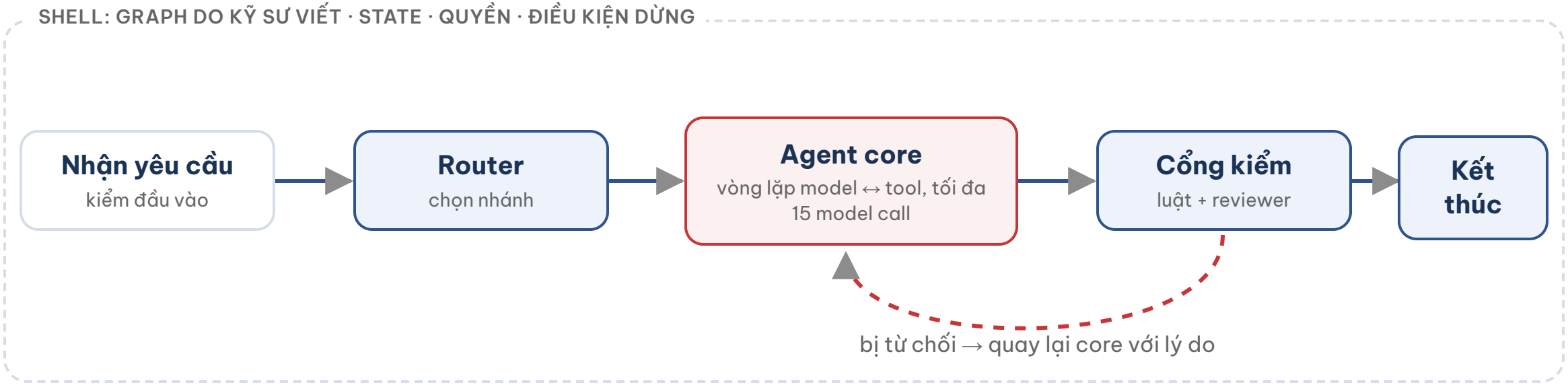
Nguồn: cùng nguồn; số nội bộ của Anthropic

**$9 → $200**

20 phút một agent so với 6 giờ harness planner–generator–evaluator, cùng một app

Nguồn: Anthropic, harness design for long-running apps (24/03/2026)

# Mẫu production phổ biến: deterministic shell, agentic core




●●●●● Vòng sửa cũng là một cạnh do kỹ sư vẽ, không phải model tự nghĩ ra.

Nguồn: Tổng hợp từ LangChain, "How to think about agent frameworks" (20/04/2025) và 12-Factor Agents, factor 8 "Own your control flow".

# Mang về từ phần 01

01 Câu hỏi thiết kế: **ai chọn bước tiếp theo**?

02 Năm mẫu workflow giải phần lớn bài toán; agent thường đắt nhất.

03 Rủi ro nhân theo số bước: cần kiểm tra và lưu trạng thái.

04 Mẫu production: deterministic shell bọc agentic core, hành động có hậu quả đi qua cổng.

VINUNIVERSITY

02

# LangGraph: state graph

Runtime để viết deterministic shell: state có kiểu, node là hàm, mọi bước lưu lại được.

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Viết một `StateGraph` có reducer, conditional edge và checkpointer.

02 Giải thích super-step và reducer.

03 Routing động bằng `Send` và `Command`.

CÂU HỎI MỞ ĐẦU Hai bước chạy song song cùng ghi vào một trường. Kết quả là của bước nào?

# Ba tầng của hệ LangChain: runtime, framework agent, harness




Tách lớp 60

Xoay -30

Chọn một lớp để xem nó làm gì.

**HỆ QUẢ**

Tầng nào cũng là graph, nên checkpoint, interrupt và stream dùng được ở mọi tầng.

Nguồn: LangChain & LangGraph 1.0 (22/10/2025) · docs.langchain.com/oss/python/deepagents/overview

# Một StateGraph có bốn phần: state, node, edge, compile

```python
class Refund(TypedDict):
    evidence: Annotated[list[str], operator.add]
    amount_vnd: int

def check_order(s): return {"evidence": ["đơn hợp lệ"]}
def route(s):
    return "review" if s["amount_vnd"] > 1_000_000 else "auto"

g = StateGraph(Refund)
g.add_node("check", check_order)
g.add_node("review", review); g.add_node("auto", auto)
g.add_edge(START, "check")
g.add_conditional_edges("check", route, ["review", "auto"])
g.add_edge("review", END); g.add_edge("auto", END)
app = g.compile(checkpointer=InMemorySaver())
app.invoke(inp, {"configurable": {"thread_id": "KN-8812"}})
```

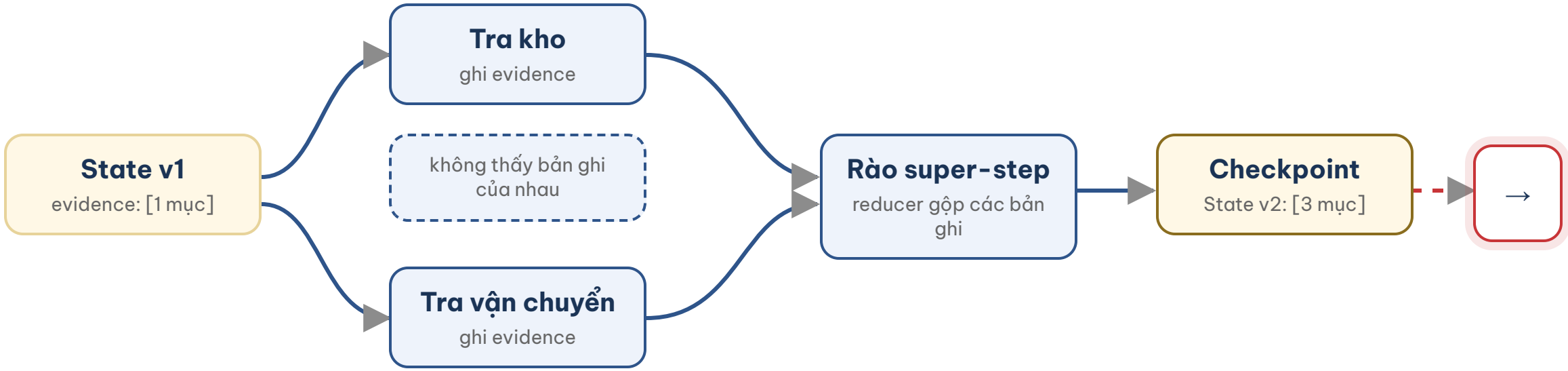
BƯỚC 4 / 4

**COMPILE**

Gắn checkpointer; `thread_id` là khoá để lưu và nối lại lần chạy.

Nguồn: docs.langchain.com/oss/python/langgraph/graph-api · đã chạy với langgraph 1.2.14

# Graph chạy theo super-step: node song song đọc cùng một bản state




Super-step tiếp theo đọc State v2.

Nguồn: Mô hình Pregel / Bulk Synchronous Parallel · docs.langchain.com/oss/python/langgraph/graph-api

# Bật reducer: hai lần ghi song song được gộp, hay báo lỗi?

```python
class S(TypedDict):
    evidence: list[str]

# hai node chạy song song trong cùng super-step
tra_kho      → {"evidence": ["thiếu 2 món"]}
tra_van_chuyen → {"evidence": ["giao 1 kiện"]}
```

**KẾT QUẢ**

`InvalidUpdateError`: INVALID_CONCURRENT_GRAPH_UPDATE — kênh không có reducer chỉ nhận một giá trị mỗi bước.

**Dùng reducer**
operator.add

**DỰ ĐOÁN TRƯỚC**

Không có reducer, hai node cùng ghi: chuyện gì xảy ra? Bật rồi tắt reducer để kiểm tra.

Với tin nhắn chat, dùng `add_messages`: nó gộp theo id, không chỉ nối danh sách.

Nguồn: docs.langchain.com/oss/python/langgraph/INVALID_CONCURRENT_GRAPH_UPDATE · graph-api (reducers)

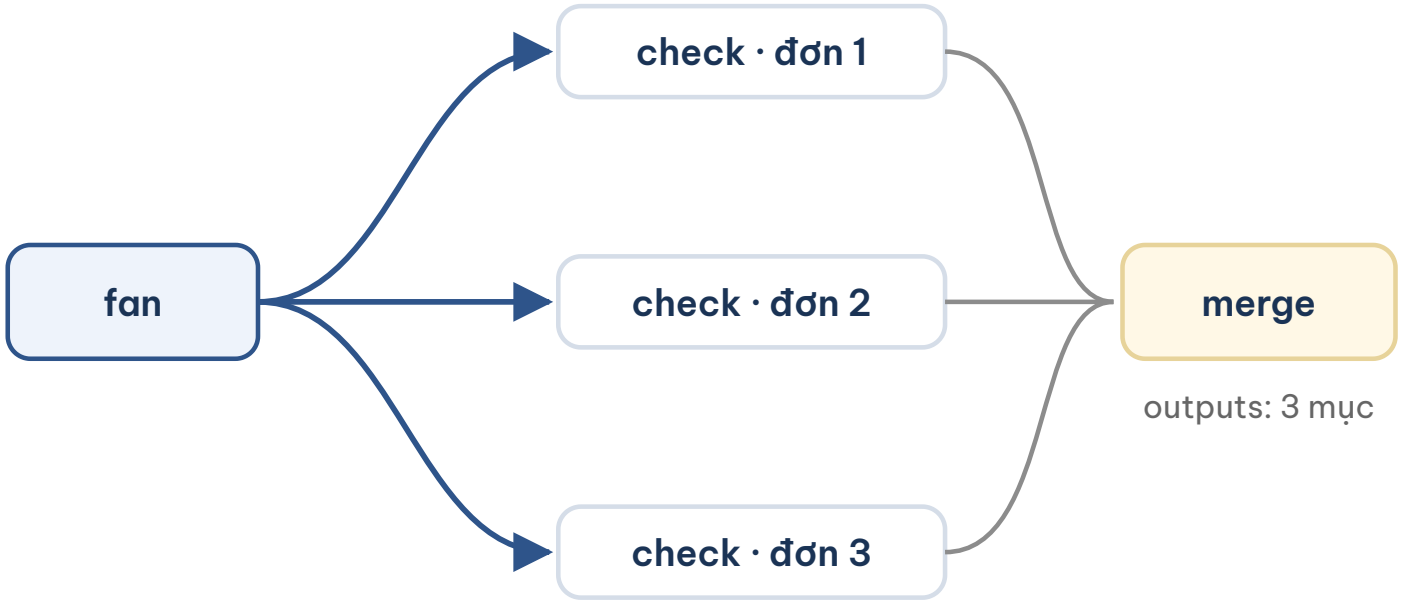
# `Send` : số nhánh quyết định lúc chạy, không vẽ trước

```python
def fan(s):
  return [Send("check",
               {"order": o})
          for o in s["orders"]]

g.add_conditional_edges(
  START, fan, ["check"])
```

Số đơn của khách 3

Mỗi `Send` mang đầu vào riêng của nhánh. Trường kết quả cần reducer để gộp.




Nguồn: docs.langchain.com/oss/python/langgraph/graph-api (Send) · đã chạy với langgraph 1.2.14

# `Command`: cập nhật state và chọn bước tiếp trong một giá trị

```python
from langgraph.types import Command

def triage(s) -> Command[Literal["fraud", "refund"]]:
    risk = score(s)
    return Command(
        update={"risk": risk},
        goto="fraud" if risk > 0.8 else "refund",
    )
```

Nguồn: docs.langchain.com/oss/python/langgraph/graph-api (Command)

**KHI NÀO DÙNG**

Quyết định routing nằm ngay trong logic của node, ví dụ một agent quyết định chuyển việc (handoff).

**SUBGRAPH**

`Command(graph=Command.PARENT, goto=…)` nhảy ra graph cha.

**HAY QUÊN**

Kiểu trả về `Literal[...]` giúp vẽ cạnh khi không có `add_edge`.

# Guardrails dựng sẵn trong graph: giới hạn, thử lại, timeout, cache

| Cơ chế | API | Lưu ý |
|---|---|---|
| Giới hạn vòng lặp | `recursion_limit` trong config | Đếm super-step, không đếm token; mặc định 1.000 từ 1.0.6 |
| Thử lại node | `retry_policy=RetryPolicy(...)` | Không thử lại lỗi lập trình như `ValueError` |
| Timeout node | `timeout` / `TimeoutPolicy` | Từ 1.2; chỉ cho node async |
| Cache node | `cache_policy=` + `compile(cache=...)` | Khoá mặc định là hash đầu vào |
| Graph con | subgraph làm node | Namespace riêng trong checkpoint và stream |

Nguồn: docs.langchain.com/oss/python/langgraph/fault-tolerance · graph-api · LangGraph 1.2.0 release (12/05/2026)

VINUNIVERSITY

# Dự đoán: agent cần 15 lượt gọi tool, chạy với `recursion_limit=25`

A Chạy xong, vì 15 < 25

B Ném lỗi giữa chừng, khoảng lượt thứ 12

C Dừng êm ở bước 25 và trả kết quả dở

D Bỏ qua giới hạn vì đây là agent

Bạn chắc đến đâu? Đoán | Khá chắc | Chắc chắn

Làm lại

Nguồn: Đo với langchain 1.4.3, model giả lập: 15 lượt tool + câu trả lời cuối = 31 super-step

# Mang về từ phần 02

01 LangGraph là runtime: state có kiểu, node trả cập nhật, cạnh routing, compile gắn checkpointer.

02 Node song song đọc cùng một bản state; ghi được gộp ở rào super-step bằng reducer.

03 `Send` và `Command` cho phép routing động mà vẫn nằm trong khung graph.

04 Giới hạn, thử lại, timeout là cấu hình — deterministic shell rẻ để viết.

VINUNIVERSITY

03

# Durable execution & human-in-the-loop

Sự cố, chờ reviewer, xem lại lần chạy: checkpoint biến cả ba thành việc bình thường.

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Chọn chế độ `durability` theo cái giá của việc mất một bước.

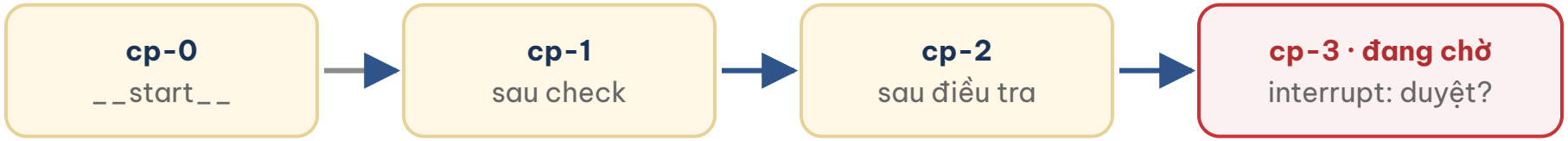
02 Dùng `interrupt()` mà không làm một side effect chạy hai lần.

03 Phân biệt checkpoint của một thread với Store dùng chung nhiều thread.

CÂU HỎI MỞ ĐẦU Server khởi động lại khi đang chờ duyệt hoàn tiền. Điều gì nên xảy ra?

# Thread là chuỗi checkpoint: mỗi super-step một checkpoint

**thread_id = "KN-8812"**




| | |
|---|---|
| **checkpoint_ns** | "" = graph gốc · "investigate:<id>" = subgraph lồng bên trong |
| **Lưu ở đâu** | InMemorySaver (dev) · SQLite · Postgres (production) · MongoDB, Cosmos DB |

Subgraph có namespace riêng; nơi lưu chọn theo môi trường.

Nguồn: docs.langchain.com/oss/python/langgraph/checkpointers

# Chế độ durability đổi độ trễ lấy khả năng khôi phục

VINUNIVERSITY

**`"exit"`**

Lưu khi graph thoát. Sập giữa chừng thì mất cả lần chạy.

việc ngắn

**`"async"` · mặc định**

Lưu trong khi bước sau chạy. Có thể mất bước cuối.

phần lớn ứng dụng

**`"sync"`**

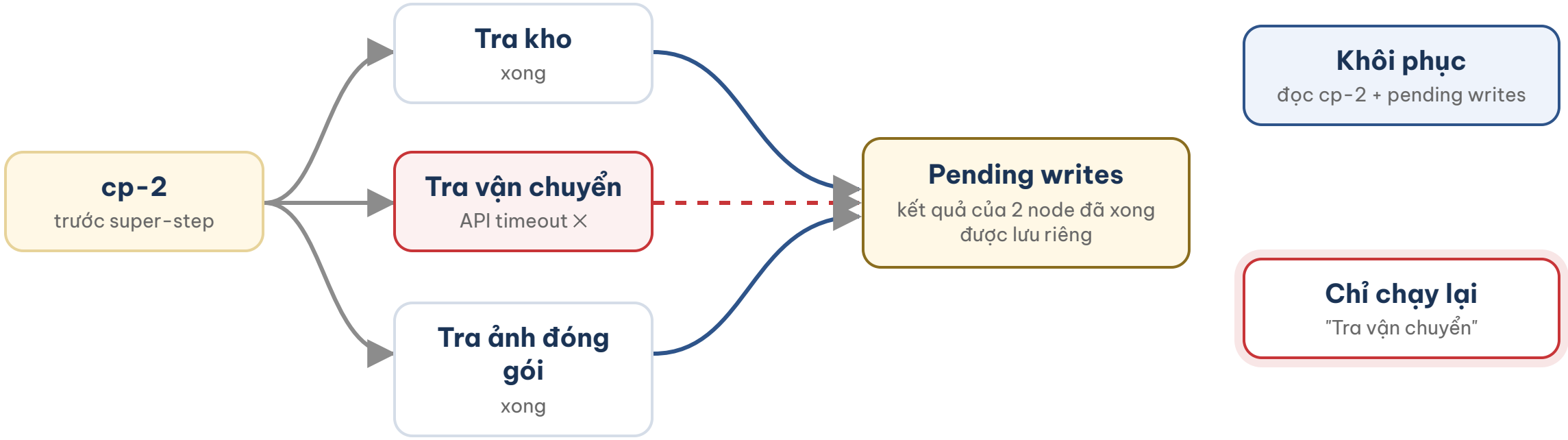
Lưu xong mới chạy bước sau. Node dở vẫn có thể chạy lại.

bước có tiền

```
app.invoke(inp, cfg, durability="sync")   # cũng có trên .stream()
```

Nguồn: langgraph/types.py `Durability` · thay cho tham số cũ `checkpoint_during`

# Node song song lỗi: khôi phục chỉ chạy lại node đó




●●●●● Hai node đã xong không bị gọi lại; chỉ node lỗi chạy lại.

Nguồn: docs.langchain.com/oss/python/langgraph/checkpointers (pending writes) · LangGraph 1.2: durable error handling qua host crash

# Resume chạy lại node từ đầu, không từ dòng interrupt

```python
def approve_refund(s):
    notify_manager(s["case_id"])        # tác dụng phụ
    ok = interrupt({"amount": s["amount"]})
    return {"approved": ok}

# lần 1: dừng ở interrupt · lần 2: người duyệt trả lời
app.invoke(Command(resume=True), cfg)
```

BƯỚC 4 / 4

**SỬA**

Đưa side effect vào node **trước**, hoặc sau khi duyệt; hoặc dùng idempotency key.

Nguồn: docs.langchain.com/oss/python/langgraph/interrupts · đã chạy với langgraph 1.2.14

# Dự đoán: đặt lệnh gọi API hoàn tiền ở đâu?

VINUNIVERSITY

A Trong cùng node, trước `interrupt()`

B Ở node riêng sau bước duyệt, kèm idempotency key

C Bất kỳ đâu, miễn là có `RetryPolicy`

D Bất kỳ đâu, miễn là `durability="sync"`

Bạn chắc đến đâu? Đoán | Khá chắc | Chắc chắn

Làm lại

# Reviewer có bốn lựa chọn, đặt theo từng tool

```python
agent = create_agent(model, tools=[refund, search],
  middleware=[HumanInTheLoopMiddleware(interrupt_on={
      "refund": {"allowed_decisions":
          ["approve", "edit", "reject", "respond"]},
          "search": False,
      })],
      checkpointer=PostgresSaver(...))

r = agent.invoke(msg, cfg)
r["__interrupt__"][0].value["action_requests"]
agent.invoke(Command(resume={"decisions": [
    {"type": "edit", "edited_action": ...}]}), cfg)
```

**QUYẾT ĐỊNH**

**approve** · **edit** (sửa tham số) · **reject** (kèm lý do) · **respond** (người trả lời thay tool).

**DỪNG KHI NÀO**

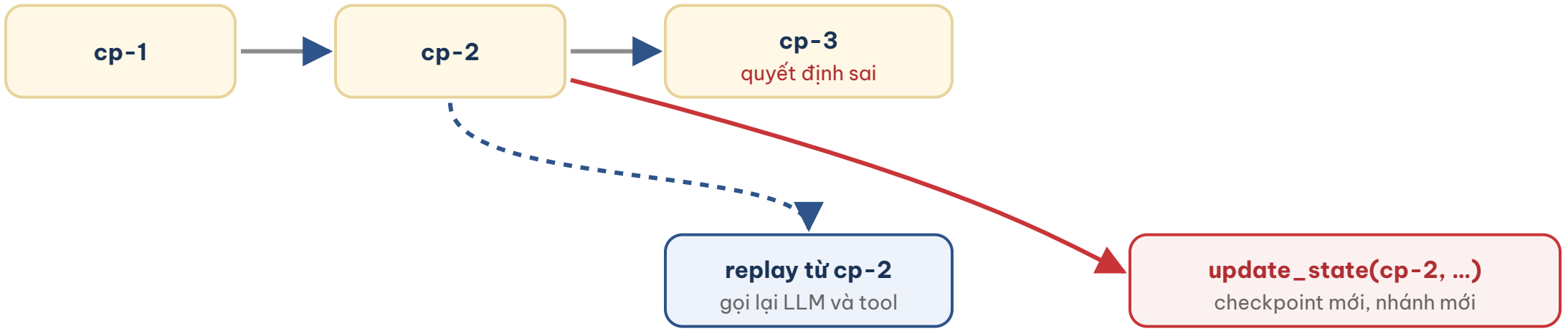
Sau khi model chọn tool, **trước khi** tool nào chạy.

**HAY NHẦM**

Một số ví dụ trong docs ghi `"modify"`; code dùng `"edit"`.

Nguồn: docs.langchain.com/oss/python/langchain/human-in-the-loop · đã chạy với langchain 1.4.3

# Time travel: chạy lại từ checkpoint cũ, hoặc fork bằng update_state




Lịch sử cũ không bị xoá: **update_state fork, không quay ngược**.

Dùng để gỡ lỗi và để reviewer sửa dữ liệu agent đã hiểu sai.

Nguồn: docs.langchain.com/oss/python/langgraph/use-time-travel · đã chạy với langgraph 1.2.14

# Đoán trước: dữ liệu này nằm trong checkpoint hay Store?

| | Checkpoint (một thread) | Store (mọi thread) |
|---|---|---|
| Tin nhắn của vụ #8812 | ? | ? |
| Khách này thích được gọi điện | ? | ? |
| Bước duyệt đang chờ quản lý | ? | ? |
| Mẹo rút ra từ 200 vụ cũ | ? | ? |
| Bằng chứng của vụ này | ? | ? |

Store: namespace + key, có tìm kiếm ngữ nghĩa.

Che lại

Nguồn: docs.langchain.com/oss/python/langgraph/stores · checkpointers

# Ra production: stream cho giao diện, server cho khách nhắn chồng

| `stream_mode` | Trả lời câu hỏi |
|---|---|
| `updates` | Node nào vừa đổi gì? |
| `messages` | Token LLM đang sinh? |
| `custom` | Tool báo tiến độ gì? |
| `values` +3 | State đầy đủ, runtime |

| Khách nhắn chồng | Hành vi |
|---|---|
| `enqueue` | Xếp hàng sau run |
| `reject` | Từ chối tin mới |
| `interrupt` | Dừng run cũ, giữ tiến độ |
| `rollback` | Huỷ run cũ |

**SERVER**

Thử bằng `langgraph dev` ; chạy thật trên LangSmith Deployment hoặc tự host.

Nguồn: docs.langchain.com/oss/python/langgraph/streaming · docs.langchain.com/langsmith/agent-server (double-texting) · LangGraph 1.1.0 (10/03/2026)

# Mang về từ phần 03

01 Thread là chuỗi checkpoint; interrupt dừng ở checkpoint, không giữ tiến trình.

02 `durability`: exit · async (mặc định) · sync — đổi độ trễ lấy khôi phục.

03 Resume chạy lại node từ đầu: side effect cần node riêng và idempotency key.

04 Checkpoint giữ thực thi của một thread; Store giữ memory dùng chung.

VINUNIVERSITY

04

# Từ agent đến Deep Agents

Middleware trả quyền kiểm soát cho kỹ sư; Deep Agents xếp sẵn middleware cho việc dài.

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Chèn hard rule vào `create_agent` bằng middleware.

02 Giải thích bốn động tác context của Deep Agents.

03 Đặt agent vào trong graph, và chỉ bật phần harness bài toán cần.

CÂU HỎI MỞ ĐẦU Chặn hoàn tiền quá 2 triệu: viết luật vào prompt hay vào code?

# `create_agent` là vòng lặp model ↔ tool, compile thành một graph

```python
from langchain.agents import create_agent

agent = create_agent(
    "anthropic:claude-sonnet-5-5",
    tools=[lookup_order, carrier_status, propose_refund],
    system_prompt="Bạn điều tra khiếu nại giao hàng.",
    middleware=[...],
    checkpointer=PostgresSaver(...),
)
agent.invoke({"messages": [...]}, cfg)
```

**BÊN TRONG**

Core loop: `model ↔ tools`, còn tool call thì quay lại. Middleware có thể thêm node.

**THAY CHO**

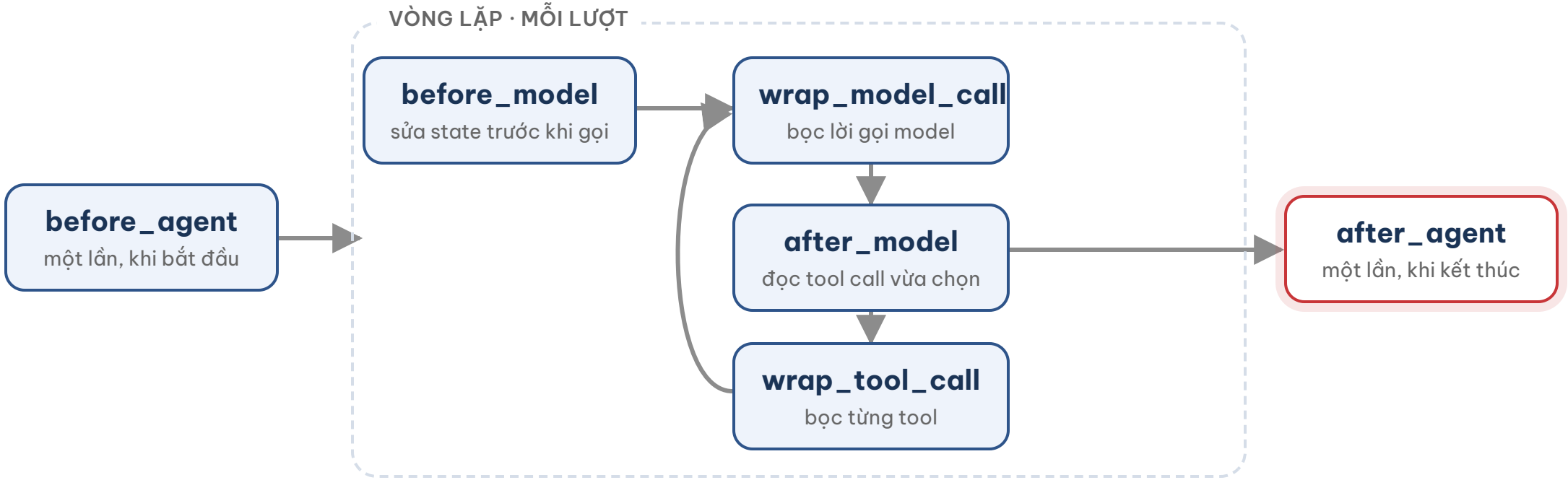
`create_react_agent` của `langgraph.prebuilt` (cũ).

**HỆ QUẢ**

Model quyết khi nào dừng. Muốn giới hạn: middleware hoặc `recursion_limit`.

Nguồn: LangChain 1.0 (22/10/2025) · docs.langchain.com/oss/python/langchain/agents

# Middleware có sáu cửa quanh vòng lặp: đúng cửa thì đúng quyền




Ghi log, lưu memory, chấm điểm lần chạy.

Nguồn: langchain/agents/middleware/types.py (1.4.3) · docs.langchain.com/oss/python/langchain/middleware

# Prompt khuyên, middleware ép: luật hoàn tiền viết thành code

```python
from langchain.agents.middleware import wrap_tool_call
from langchain.messages import ToolMessage

@wrap_tool_call
def refund_cap(request, handler):
    call = request.tool_call
    if (call["name"] == "propose_refund"
            and call["args"]["amount"] > 2_000_000):
        return ToolMessage("Vượt hạn mức.",
                           tool_call_id=call["id"])
    return handler(request)

agent = create_agent(model, tools, middleware=[refund_cap])
```

**VÌ SAO HIỆU QUẢ**

Model vẫn chọn; code quyết định lựa chọn đó có được chạy không.

**AGENT THẤY GÌ**

Một ToolMessage giải thích lý do, nên nó có thể đổi hướng.

**ĐỪNG**

Ném exception: mặc định nó làm dừng cả lần chạy.

Nguồn: docs.langchain.com/oss/python/langchain/middleware (custom middleware, decorators)

# Ba cách ghép graph và agent

## Agent trong một node

Một node gọi agent, nhận kết quả có kiểu.

```
def investigate(s):
    out = agent.invoke(
      {"messages": s["messages"]})
    return {"proposal":
      out["structured_response"]}
```

## Workflow làm tool

Agent chọn **khi nào** chạy quy trình; graph quyết **làm thế nào**.

## Graph làm subagent

`CompiledSubAgent(..., runnable=graph)` ; state cần khoá `messages` .

**Phần biết trước viết thành graph; phần mở để model chọn.**

Nguồn: docs.langchain.com/oss/python/deepagents/subagents · cả ba mẫu đã chạy với langgraph 1.2.14, deepagents 0.7.23

# Agent "nông" hỏng ở việc dài theo sáu kiểu quen thuộc

### Kết quả tool quá lớn
→ ghi ra tệp, giữ preview

### Lịch sử vượt context window
→ tóm tắt, lưu bản gốc

### Không có chỗ nháp
→ hệ thống virtual filesystem

### Việc phụ làm đầy context
→ subagent context riêng

### Mất kế hoạch
→ danh sách todo (tuỳ chọn)

### Tool call bị bỏ dở
→ vá bằng ToolMessage

Nguồn: docs.langchain.com/oss/python/deepagents/overview

# `create_deep_agent` = `create_agent` + một chồng middleware xếp sẵn

- **Filesystem** — tool tệp; kết quả > 20k token ra tệp
- **SubAgent** — tool `task`, context riêng
- **Summarization** (85%) · **PatchToolCalls**
- *middleware của bạn chèn ở đây*
- **Skills** · **Caching** · **Memory** · **HITL** — khi được bật

**TỪ 0.7**

Không còn `write_todos` mặc định. Muốn planning: `middleware=[TodoListMiddleware()]`.

Nguồn: deepagents 0.7.23, graph.py (thứ tự chồng middleware) · docs.langchain.com/oss/python/deepagents/overview ("task planning is opt-in")

# Mười dòng là một deep agent, và nó trả về cả state

```python
from deepagents import create_deep_agent

agent = create_deep_agent(
    model="anthropic:claude-sonnet-5-5",
    tools=[search_logs, carrier_status],
    system_prompt="Bạn điều tra khiếu nại giao hàng.",
    subagents=[{"name": "log-reader",
                "description": "Đọc log kho",
                "system_prompt": "Báo mốc giờ."}],
    interrupt_on={"write_file": True},
    checkpointer=InMemorySaver(),
)
r = agent.invoke({"messages": [...]}, cfg)
r["messages"], r["files"]
```

**TRẢ VỀ**

Một graph LangGraph đã compile: checkpoint, stream, interrupt dùng nguyên.

**STATE**

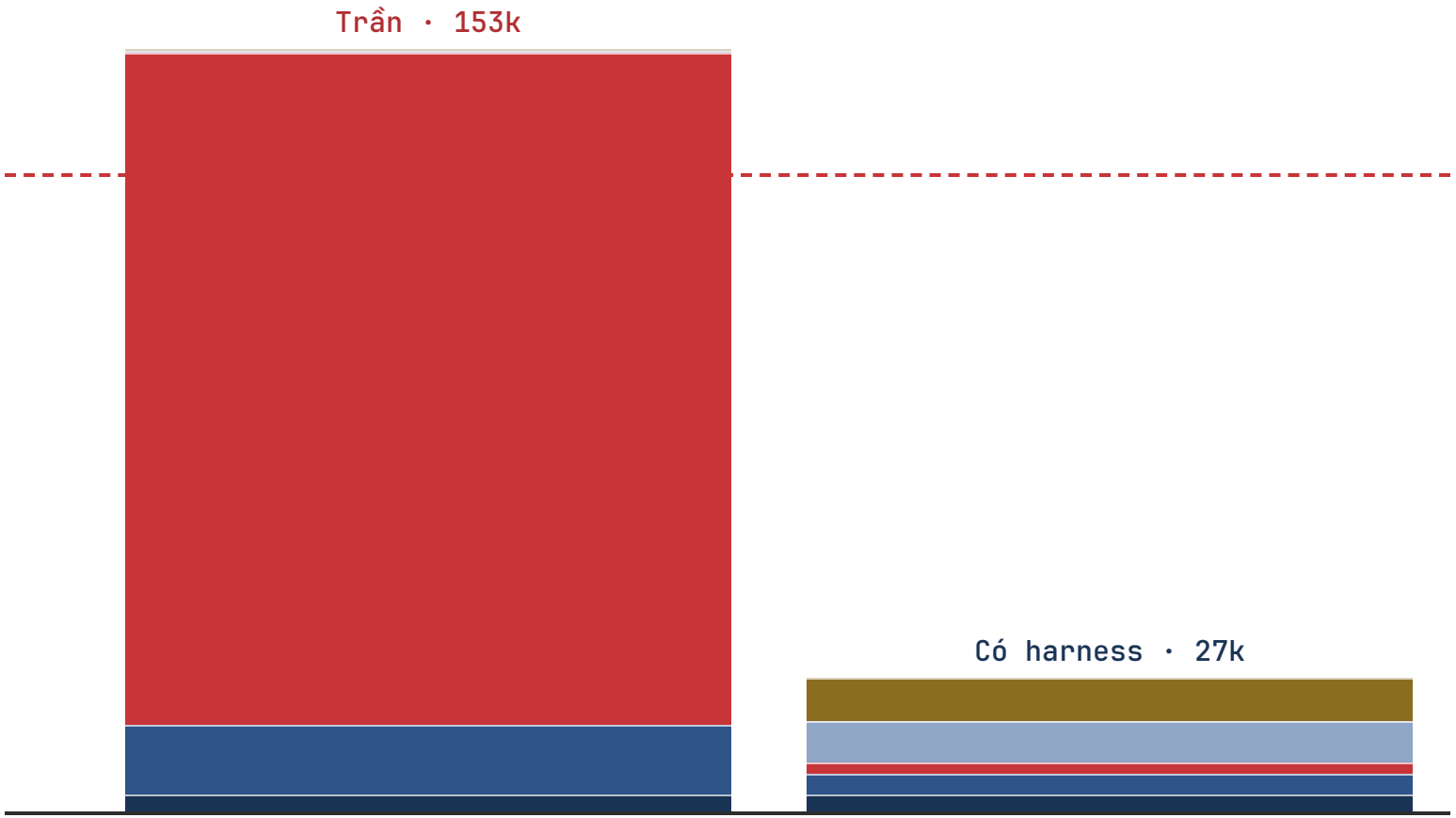
`messages` và `files` (virtual filesystem); thêm `todos` khi bật TodoList.

**HAY QUÊN**

`interrupt_on` cần checkpointer và `thread_id`.

Nguồn: docs.langchain.com/oss/python/deepagents/overview · đã chạy với deepagents 0.7.23 (model giả lập)

# Bật từng cơ chế: write, select, compress, isolate context




**Write: ra tệp**
kết quả > 20k token

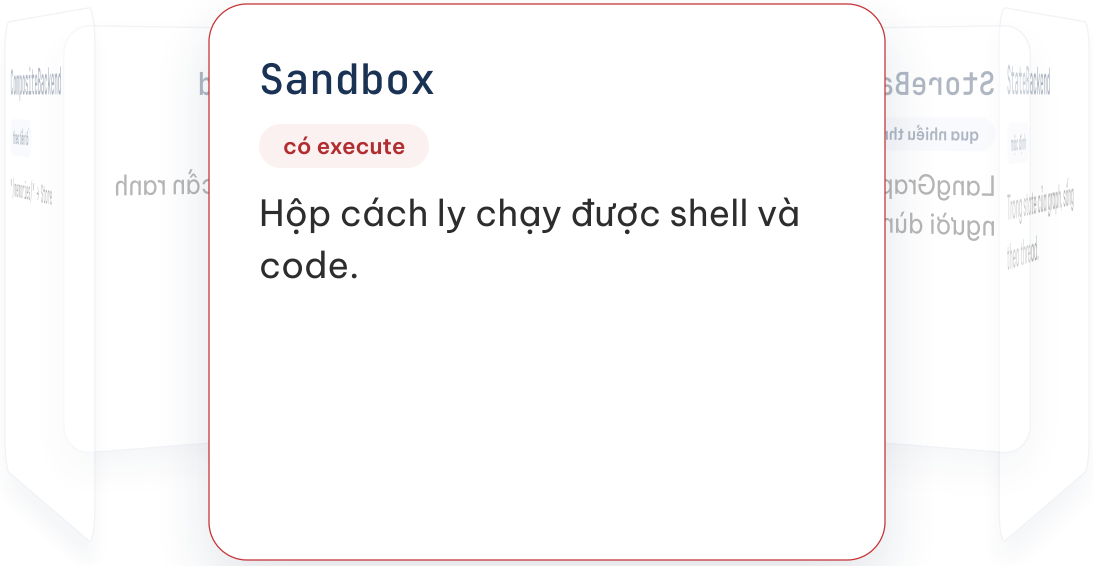
**Isolate: subagent**
chỉ kết quả cuối quay về

**Compress ở 85%**
giữ 10% gần nhất

- System prompt + tool
- Tin nhắn
- Kết quả tool
- Preview / kết quả subagent
- Summary

**Minh hoạ** · mô hình đồ chơi: cột trái luôn là vòng lặp trần; vạch đỏ là context window 128k. Mặc định deepagents 0.7.23, cấu hình được: 20.000 token (filesystem.py); 85% / 10% context window khi model có max_input_tokens, nếu không 170k token / 6 tin (summarization.py).

# Agent thấy đường dẫn; backend quyết định tệp nằm ở đâu




**ĐOÁN TRƯỚC**

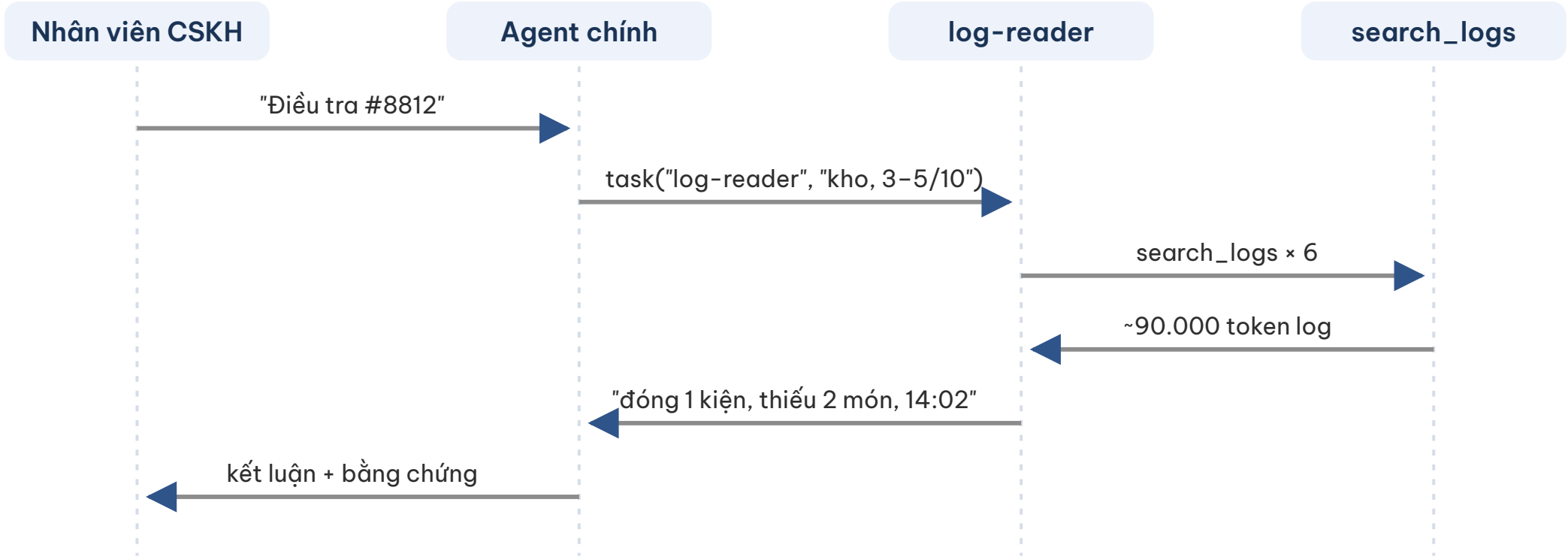
Nhớ rằng khách này hay khiếu nại sai qua nhiều vụ: backend nào?

← Xoay  Xoay →

BƯỚC 4 / 4

Nguồn: deepagents/backends: state, store, filesystem, local_shell, composite, sandbox (0.7.23)

# Subagent giữ kết quả lớn ở ngoài; chỉ thông điệp cuối quay về




Context chính vẫn gọn để kết luận.

**Minh hoạ** · số token là ví dụ.

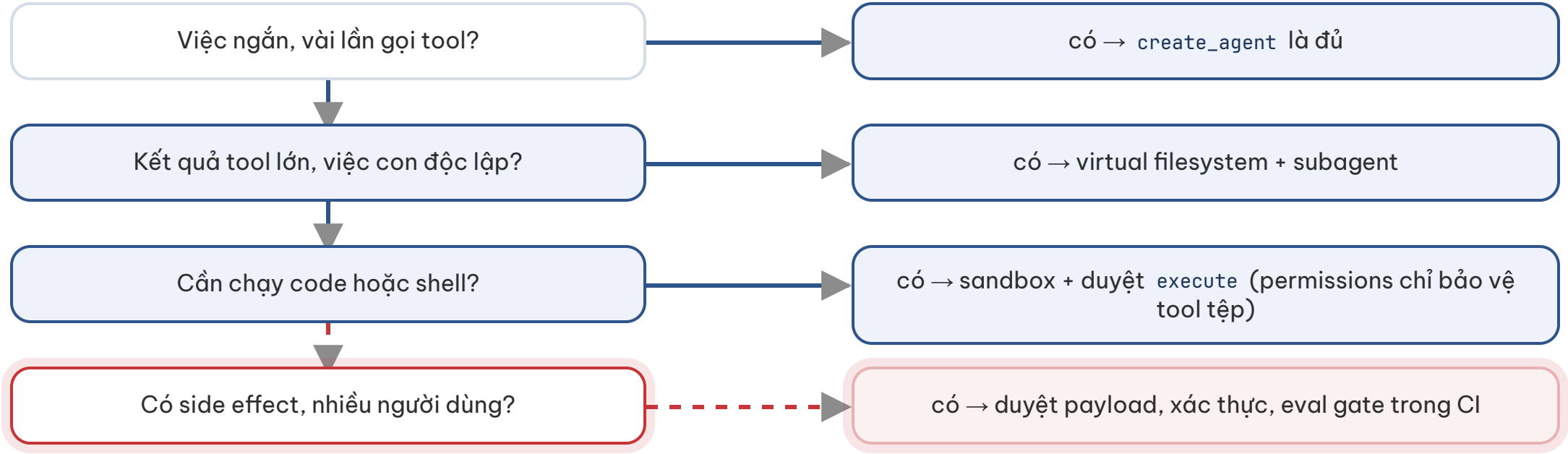
# Memory, skill, tool, subagent: khác nhau ở lúc được nạp

| Thành phần | Nạp khi nào | Dùng cho |
|---|---|---|
| Memory ( `AGENTS.md` ) | Luôn trong prompt (khi bật) | Quy ước bền vững |
| Skill ( `SKILL.md` ) | Mô tả trước, nội dung khi cần | Quy trình dài, ít dùng |
| Tool | Schema mỗi lượt (tool gắn skill: khi skill được đọc) | Hành động thường xuyên |
| Subagent | Mô tả; chạy riêng khi gọi | Việc nặng |

**Chọn thứ rẻ nhất đủ dùng.**

Nguồn: docs.langchain.com/oss/python/deepagents (skills, memory, subagents)

# Cây quyết định: chỉ thêm phần harness khi bài toán cần




Nguồn: docs.langchain.com/oss/python/deepagents/overview

# Mang về từ phần 04

01 `create_agent` là graph: core loop model ↔ tools; model quyết khi nào dừng.

02 Hard rule viết trong middleware (`wrap_tool_call`), không trong prompt.

03 Deep Agents = `create_agent` + chồng middleware xếp sẵn: tệp, subagent, tóm tắt.

04 Context: write ra tệp, select khi cần, compress, isolate bằng subagent.

05 Ghép: agent trong node, workflow làm tool, graph làm subagent.

VINUNIVERSITY

05

# Bản đồ framework 2026

Ba câu hỏi cho mọi framework: ai chọn bước, state ở đâu, ai vận hành.

SAU PHẦN NÀY, BẠN CÓ THỂ

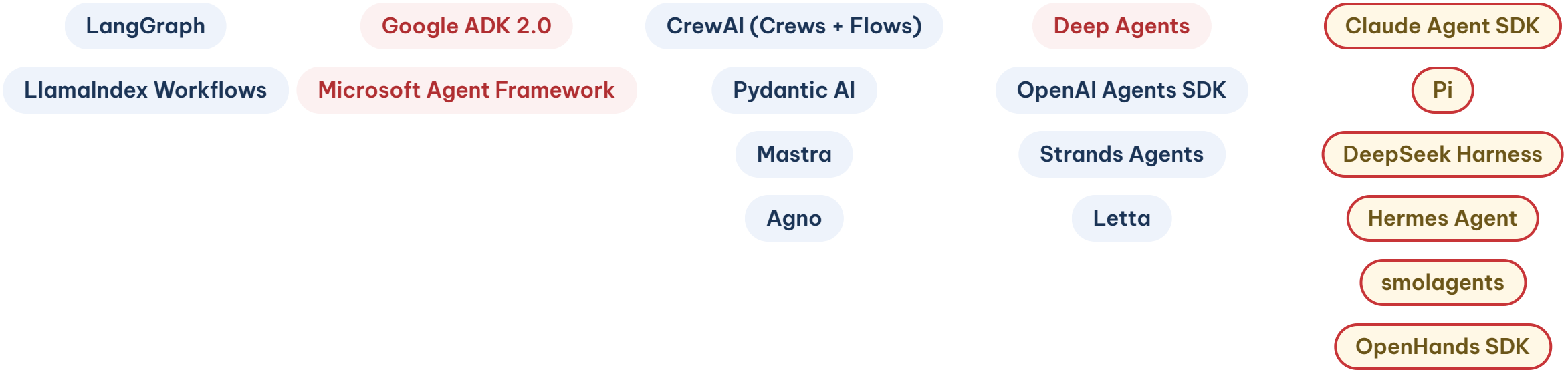
01 Đặt một framework mới lên phổ.

02 So sánh ADK, MAF, OpenAI/Claude SDK, Pi, DeepSeek, Hermes.

03 Chọn shortlist và biết cần thử gì trước khi chốt.

CÂU HỎI MỞ ĐẦU Framework nào "tốt nhất"? Câu hỏi này sai ở đâu?

# Framework trên phổ: graph bên trái, vòng lặp mở bên phải




●●●●● Harness hoàn chỉnh: model tự chọn gần như mọi bước. Màu vàng: sản phẩm/harness hơn là thư viện.

Nguồn: Vị trí trên phổ là đánh giá của bài giảng theo mặc định của từng framework; nhiều framework làm được cả hai đầu. Nguồn: tài liệu và release chính thức, đọc 08/10/2026.

# Google ADK 2.0: graph Workflow làm trung tâm, agent là một node

- 2.0 GA 19/05/2026; Python, Java, Go, TS.
- `Workflow`: routing, fan-out, vòng lặp, reviewer.
- Vertex AI Agent Engine, Cloud Run; MCP, A2A.

```
from google.adk import Agent
root_agent = Agent(model="gemini-3.5-flash",
    name="hello", instruction="Trả lời ngắn gọn.")
```

**ĐÁNG DẠY**

Google tạo ra A2A (04/2025).

**CẨN THẬN**

Resume bằng replay sự kiện: side effect cần idempotency.

Nguồn: github.com/google/adk-python releases (2.11.0, 01/10/2026) · google/adk-docs 2.0 · a2a-protocol.org blog (27/08/2026)

VINUNIVERSITY

# Microsoft Agent Framework thay AutoGen và Semantic Kernel bằng workflow có checkpoint

- 1.0 GA 04/2026; Python, .NET; MIT.
- **Workflow**: executor + edge, checkpoint theo super-step.
- AutoGen chuyển sang chế độ bảo trì.
- MCP, A2A, AG-UI; Foundry hoặc tự host.

```
agent = Agent(client=FoundryChatClient(credential=cred),
              name="Hello", instructions="Trả lời ngắn.")
print((await agent.run("Chào")).text)
```

**ĐÁNG DẠY**

Cùng mô hình super-step như LangGraph.

**CẨN THẬN**

Checkpoint dùng pickle: bảo vệ kho lưu.

Nguồn: devblogs.microsoft.com/agent-framework (GA, 04/2026) · github.com/microsoft/agent-framework · github.com/microsoft/autogen (maintenance mode)

# OpenAI Agents SDK đưa primitive; Claude Agent SDK đưa harness

### OPENAI AGENTS SDK · 0.X

- **Handoff** chuyển quyền; **agent-as-tool** giữ quyền
- Guardrail, session, tracing

### CLAUDE AGENT SDK · 0.X

- "Claude Code dạng thư viện"
- Hook, quyền, subagent, skill

```
result = await Runner.run(Agent(name="Hello", instructions="Ngắn gọn."), "Chào")   # OpenAI
async for msg in query(prompt="Chào"): print(msg)                                  # Claude
```

Nguồn: github.com/openai/openai-agents-python · developers.openai.com (Agent Builder) · anthropic.com/news/enabling-claude-code-to-work-more-autonomously · github.com/anthropics/claude-agent-sdk-python

# Pi: harness tối giản, chính sách để host tự viết

VINUNIVERSITY

### PI · MARIO ZECHNER · MIT · TYPESCRIPT

- Prompt ngắn, ít tool: đọc, ghi, sửa, shell
- Không có sẵn quyền hay subagent
- 1.0 ra 01/10/2026

### DEEP AGENTS · HARNESS ĐẦY ĐỦ

- Nhiều middleware dựng sẵn
- Ít việc phải tự viết
- Nhiều thứ phải hiểu khi gỡ lỗi

**Chính sách nằm ở harness hay ở ứng dụng của bạn?**

Nguồn: github.com/earendil-works/pi (trước là badlogic/pi-mono; 1.1.0, 07/10/2026) · package @earendil-works/pi-coding-agent

# Hai harness mở mới: DeepSeek và Hermes Agent

VINUNIVERSITY

### DEEPSEEK: DSH + API MODEL

- **dsh**: TypeScript, MIT, mọi thứ là plugin; **preview**
- API V4: suy nghĩ khi gọi tool, tương thích Anthropic
- Phải gửi lại `reasoning_content`

### HERMES AGENT (NOUS RESEARCH)

- Tự viết skill sau việc khó
- Memory FTS5; gateway nhiều kênh
- Sản phẩm, không phải thư viện

### CẨN THẬN

Skill tự sinh cần reviewer trước khi chạy với quyền thật.

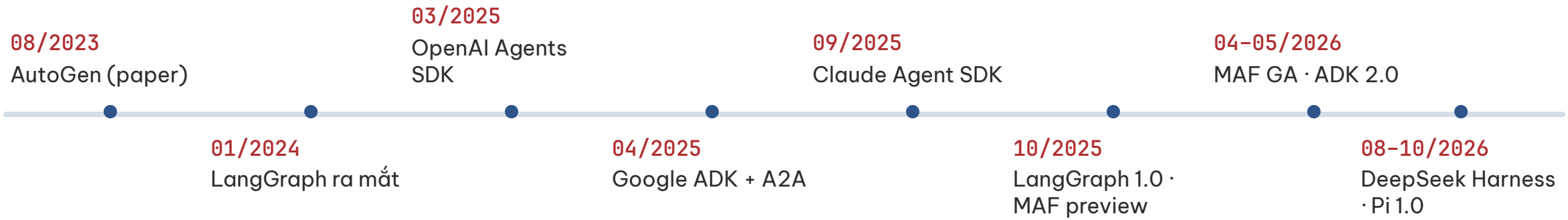
Nguồn: github.com/deepseek-ai/deepseek-harness · api-docs.deepseek.com/guides/anthropic_api · github.com/NousResearch/hermes-agent (v0.21.x)

# Các framework còn lại, đọc theo một cột: orchestration bằng gì

| Framework | Orchestration | Bền / khôi phục |
|---|---|---|
| CrewAI | Crew (vai) + Flow (sự kiện) | State của Flow |
| Pydantic AI | Agent có kiểu + graph | Qua Temporal, DBOS |
| Mastra | Agent + Workflow (TS) | Suspend/resume |
| Strands (AWS) | Vòng lặp trước, graph tuỳ chọn | Session |
| smolagents | CodeAct: code là hành động | Không |
| LlamaIndex Workflows | Bước theo sự kiện | Checkpoint |
| Agno · Letta · OpenHands | Nền tảng · memory · coding | Session / state |

Nguồn: README và tài liệu chính thức của từng dự án, đọc 08/10/2026; chi tiết và link trong _research/L4-framework-landscape.md

# Ba năm: từ hội thoại đa agent đến graph bền và harness




2026: graph bền lên GA; harness mã nguồn mở bùng nổ.

Nguồn: arXiv 2308.08155 · langchain.com/blog · openai.com/index/new-tools-for-building-agents · developers.googleblog.com · anthropic.com/news · devblogs.microsoft.com · GitHub releases

VINUNIVERSITY

# Dự đoán: hỏi agent của hãng vận chuyển bằng protocol nào?

A MCP

B A2A

C AG-UI

D ACP

Bạn chắc đến đâu? Đoán | Khá chắc | Chắc chắn

Làm lại

Nguồn: a2a-protocol.org · modelcontextprotocol.io · docs.ag-ui.com · agentclientprotocol.com

# Đoán trước: mỗi đội đứng đâu trên phổ, thử framework nào?

| | Điểm trên phổ | Shortlist để thử |
|---|---|---|
| Ngân hàng: duyệt khoản vay | ? | ? |
| Startup TypeScript: trợ lý CSKH | ? | ? |
| Coding agent cho repo nội bộ | ? | ? |
| Trợ lý cá nhân qua app nhắn tin | ? | ? |

Thử hai ứng viên trên cùng ba việc trước khi chốt.

Che lại

# Mang về từ phần 05

01 Ba câu hỏi: ai chọn bước, state ở đâu, ai vận hành.

02 ADK 2.0 và Microsoft Agent Framework đều đặt graph vào trung tâm.

03 Harness mở khác nhau ở chỗ chính sách nằm đâu.

04 Chọn điểm trên phổ trước, rồi thử hai ứng viên.

VINUNIVERSITY

06

# Multi-agent và frontier orchestration

Thêm agent có thể giúp nhiều hoặc hại nhiều. Câu hỏi tiếp: học được cách điều phối không?

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Dự đoán bài toán nào được lợi từ nhiều agent, bài toán nào bị hại.

02 Áp dụng "một người ghi, nhiều người đọc".

03 Xếp một bài báo vào ba tầng: thiết kế, lúc chạy, lúc thực thi.

CÂU HỎI MỞ ĐẦU Bốn agent cùng sửa một tệp cấu hình. Chuyện gì xảy ra?

# Cùng tháng 6/2025, hai bài viết kết luận ngược nhau, và cả hai đúng

**COGNITION · "DON'T BUILD MULTI-AGENTS"**

- Agent song song thiếu context chung thì quyết định mâu thuẫn
- Ưu tiên một agent, một luồng

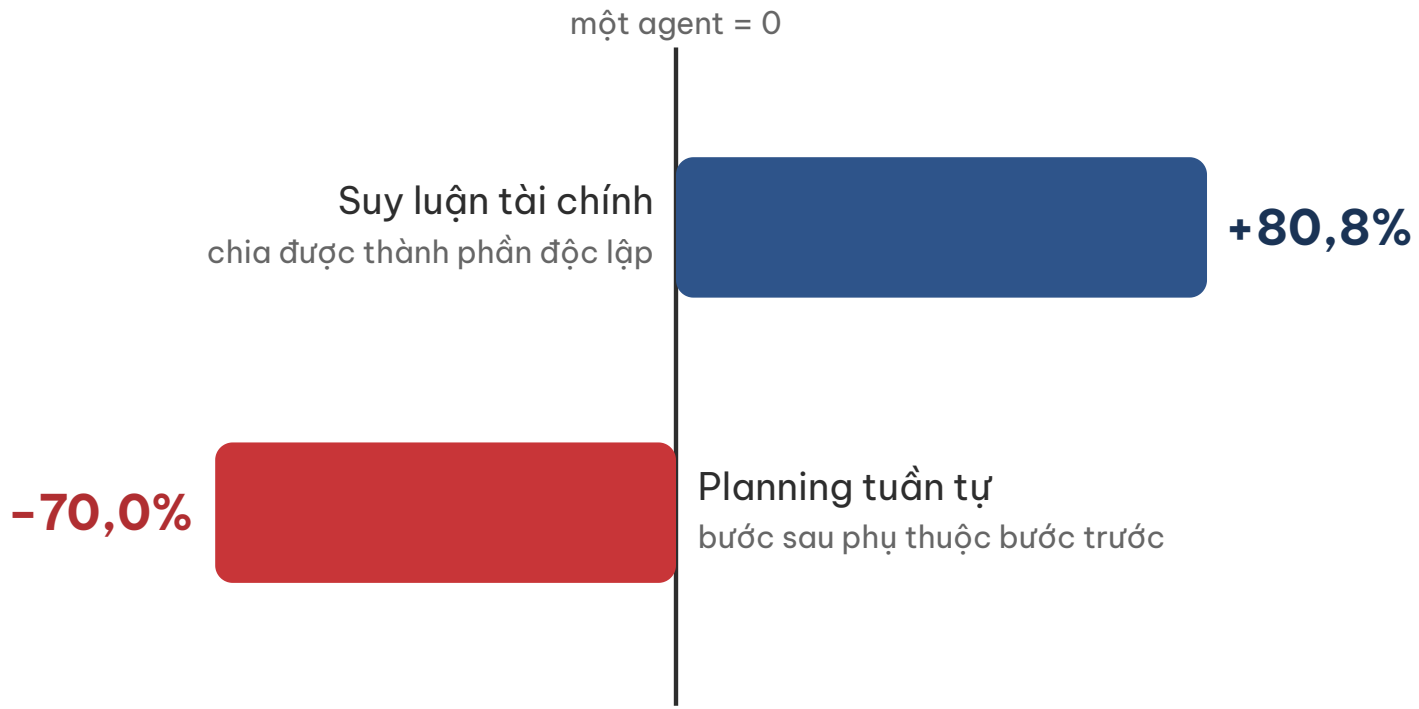
**ANTHROPIC · MULTI-AGENT RESEARCH SYSTEM**

- Opus 4 điều phối + Sonnet 4 làm worker tốt hơn Opus 4 chạy một mình **90,2%** (đánh giá nội bộ)
- Hợp với nghiên cứu nhiều nhánh

**Khác nhau ở hình dạng bài toán.**

Nguồn: cognition.com/blog/dont-build-multi-agents (12/06/2025) · anthropic.com/engineering/multi-agent-research-system (13/06/2025) — số nội bộ của Anthropic

# Thêm agent: +80,8% hoặc −70,0%, tuỳ hình dạng bài toán




**260**

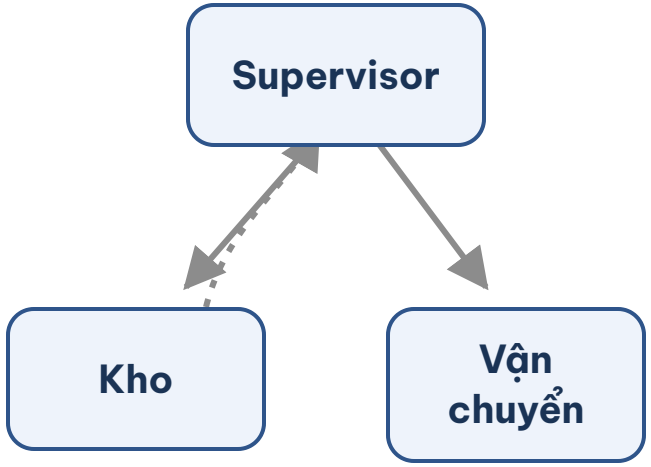
cấu hình trên 6 benchmark, 5 kiến trúc, 3 họ LLM

**BÀI HỌC**

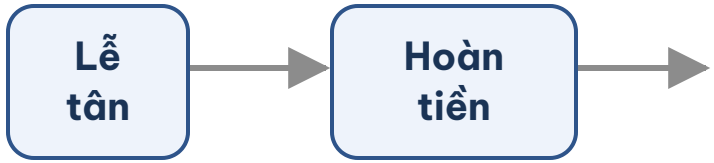
Chọn kiến trúc theo cấu trúc phụ thuộc của việc, không theo số agent.

Nguồn: Kim et al. (Google Research), "Towards a Science of Scaling Agent Systems", arXiv 2512.08296 (09/12/2025, sửa 08/04/2026)

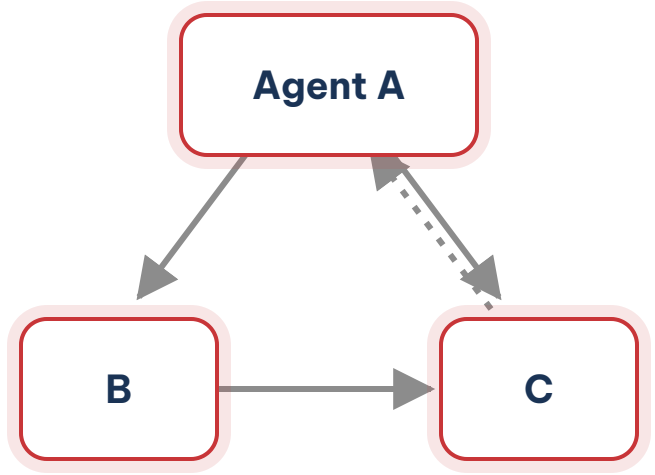
# Supervisor, handoff, swarm khác nhau ở chỗ ai giữ quyền điều khiển




Agent chuyên trách là **tool**; supervisor giữ cuộc hội thoại và gộp kết quả.




**Chuyển hẳn** quyền điều khiển (`Command(goto=…)`); agent nhận việc nói chuyện với khách.




Ngang hàng, không ai điều phối. Cần điều kiện dừng tường minh.

●●●● Swarm: linh hoạt nhất, khó gỡ lỗi nhất.

**Hỏi lớp** · Ai chịu trách nhiệm câu trả lời cuối trong mỗi mẫu?

Nguồn: OpenAI, "A practical guide to building agents" · Google Cloud, agentic design patterns (05/2026) · LangChain multi-agent docs

# Multi-agent hỏng theo 14 kiểu, và rất khó tìm ra thủ phạm

### Thiết kế hệ thống

Đặc tả thiếu, vai mờ, không biết khi nào dừng.

### Lệch giữa các agent

Giấu thông tin, bỏ qua agent khác.

### Kiểm tra và kết thúc

Dừng sớm, không kiểm hoặc kiểm sai.

**1.642**

trace gán nhãn (MAST)

**53,5%**

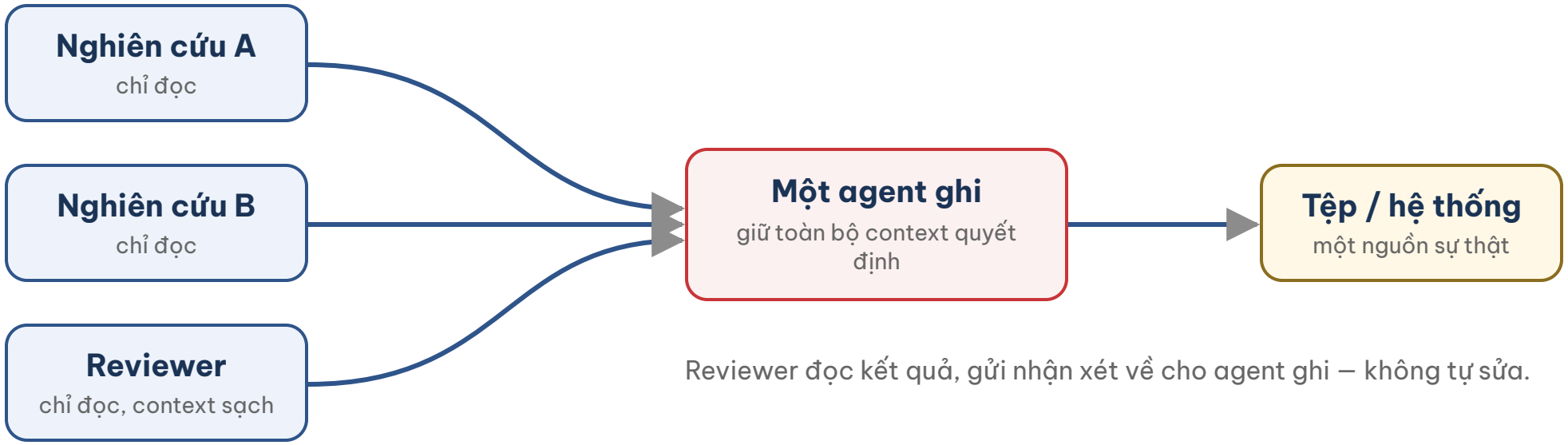
đoán đúng agent gây lỗi

**14,2%**

đoán đúng bước gây lỗi

Nguồn: Cemri et al., MAST, arXiv 2503.13657 · Who&When, arXiv 2505.00212 (127 hệ có log lỗi)

# Bắt đầu an toàn: nhiều agent đọc, một agent ghi




Reviewer đọc kết quả, gửi nhận xét về cho agent ghi — không tự sửa.

Review bằng context sạch bắt được lỗi người viết không thấy.

Nguồn: Cognition, "Multi-Agents: What's Actually Working" (22/04/2026) · Anthropic, "Patterns and problems in emerging multiagent systems" (13/08/2026)

# Nói trước, lật sau: ba niềm tin sai về multi-agent

Phương pháp tốt nhất chỉ đoán đúng bước gãy lỗi 14,2%.

**Hay nhầm:** Cần checkpoint theo bước và một chủ sở hữu.

Không chắc: tên vai không đảm bảo chuyên môn.

**Hay nhầm:** Cần tool, dữ liệu và kiểm tra.

Sai với việc tuần tự: −70% trong nghiên cứu của Google.

**Hay nhầm:** Đếm phụ thuộc trước khi đếm agent.

**Thảo luận với bạn bên cạnh** 0:45

Úp lại

# Ba tầng nghiên cứu: thiết kế, lúc chạy, lúc thực thi

VINUNIVERSITY

### 1 · LÚC THIẾT KẾ

**Tìm cấu trúc workflow**
tối ưu trước khi chạy

AFlow · MASS · DGM

### 2 · LÚC CHẠY

**Orchestrator được huấn luyện**
model nhỏ chọn worker và tool

Conductor · ToolOrchestra · MaAS

### 3 · LÚC THỰC THI

**Ràng buộc, compile, replay**
luật quanh lựa chọn của model

Agent-C · TraceCompiler · Chronicle

●●●● Đặt phần deterministic quanh lựa chọn của model.

Nguồn: Khung ba tầng là tổng hợp của bài giảng · arXiv 2410.10762, 2512.04388, 2608.02680, 2609.20625

# Learned orchestration: máy tìm cấu trúc, model nhỏ chọn worker

| Công trình | Học cái gì | Kết quả (tác giả báo cáo) |
|---|---|---|
| AFlow | Workflow dạng code | +5,7% trên 6 bộ dữ liệu |
| Darwin Gödel Machine | Agent tự sửa mã của mình | SWE-bench 20 → 50% |
| MaAS | Kiến trúc theo từng câu hỏi | 6–45% chi phí suy luận |
| Conductor · 7B | Viết workflow bằng RL | 72,35% vs 62,13% (MoA) |
| ToolOrchestra · 8B | Cân chất lượng và chi phí | HLE 37,1% vs 35,1% (GPT-5) |

Nguồn: arXiv 2410.10762 · 2505.22954 · 2502.04180 · 2512.04388 · 2511.21689

# Đi ngược trên phổ: compile trace agent thành workflow

**Nhiều trace agent**
cùng một loại việc

→

**Tìm phụ thuộc**
giá trị nào sinh ra từ bước nào

→

**Chỉ nối khi chắc**
mơ hồ → để model quyết

→

**Workflow**
phần lớn deterministic

## 34 → 11

tool call trên một luồng Venmo; nhưng chỉ 15/21 ca đạt

## 0,928 / 0,943

precision / recall trên 15.775 cạnh def-use

●●●●● Không miễn phí: compile có thể làm mất ca hiếm.

Nguồn: El Yadouni & Li, "TraceCompiler", arXiv 2608.02680 (03/08/2026) — tiền ấn phẩm, số do tác giả báo cáo

# Dự đoán: chỉ đổi harness, Terminal-Bench 2.0 từ 52,8% lên 66,5%

A Model tốt hơn 13,7 điểm

B Điểm benchmark thuộc về cả hệ thống, không riêng model

C Harness này sẽ tốt hơn 13,7 điểm với mọi model

D Agent chạy tự động được lâu hơn

Bạn chắc đến đâu? Đoán | Khá chắc | Chắc chắn

Làm lại

Nguồn: tbench.ai/benchmarks · LangChain blog, harness engineering (số tự báo cáo, 89 việc, GPT-5.2-Codex) · bảng 4.0 (28/08/2026): 64,8% = Opus 5.5 + Claude Code · metr.org/time-horizons (cập nhật 08/05/2026)

# Năm câu hỏi mở để thảo luận

01 Khi nào nhiều agent đáng chi phí?

02 Failure attribution: đúng agent, đúng bước trên trace thật?

03 Quyết định nào học, quyết định nào cố định?

04 "Tái lập được" nghĩa là gì với agent?

05 Đánh giá agent chạy dài mà không bị rò rỉ benchmark?

**Nhóm 3 người · 2 phút** 2:00

Nguồn: Tổng hợp từ arXiv 2512.08296, 2607.09996, 2512.23738, 2609.20625 và báo cáo METR

# Mang về từ phần 06

01 Multi-agent thường giúp việc chia nhánh độc lập, dễ hại việc tuần tự.

02 Bắt đầu bằng supervisor: nhiều agent đọc, một agent ghi.

03 Failure attribution: cho đúng agent, đúng bước vẫn rất khó.

04 Phổ đi hai chiều: trace agent compile được thành workflow.

05 Điểm benchmark chỉ có nghĩa khi kèm harness, budget, phiên bản.

VINUNIVERSITY

07

# Hero: dựng hệ thống lai

Ghép mọi thứ: LangGraph shell, deep agent core, approval gate bền, đánh giá trước khi ship.

SAU PHẦN NÀY, BẠN CÓ THỂ

01 Vẽ kiến trúc lai và chỉ ra ranh giới deterministic ↔ agentic.

02 Dựng Deep Research Agent trong lab: lead, subagent, sandbox, validator.

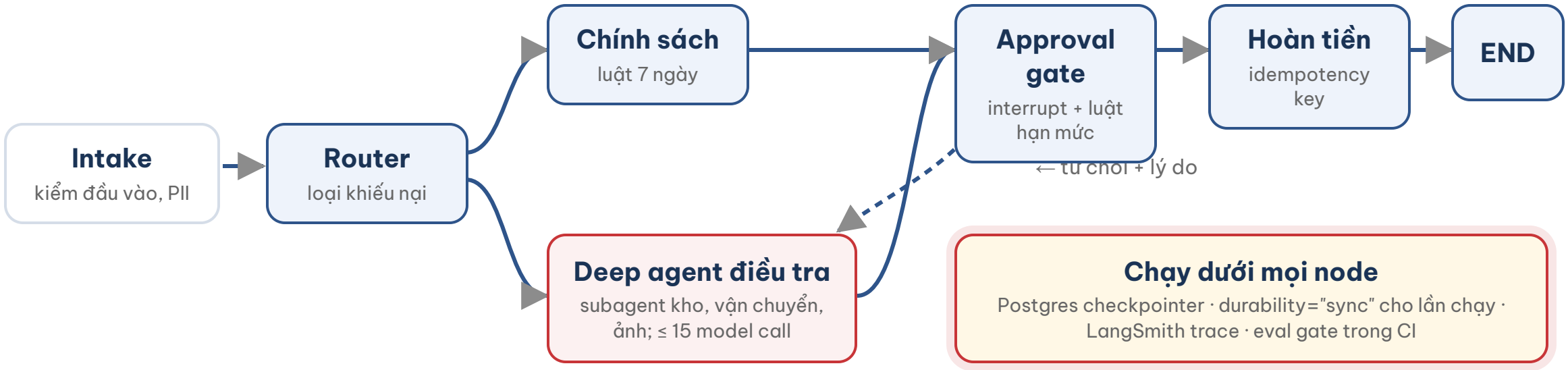
03 Kiểm hệ thống theo danh sách trước khi ship.

CÂU HỎI MỞ ĐẦU Nếu chỉ được giữ một cơ chế từ buổi hôm nay, bạn giữ cơ chế nào?

# Cùng khiếu nại #8812, hai cách chạy: graph LangGraph và deep agent

| | Graph LangGraph | Deep agent |
|---|---|---|
| **Đường đi** | vẽ trước, đếm được | model chọn từng lượt |
| **Chi phí** | ổn định, ít token | dao động, nhiều token |
| **Audit** | đọc graph là biết | phải đọc trace |
| **Hỏng kiểu** | ca chưa vẽ thì kẹt | lạc hướng, lặp, vượt budget |
| **Kỹ sư giữ** | từng nhánh | harness: tool, quyền, budget |

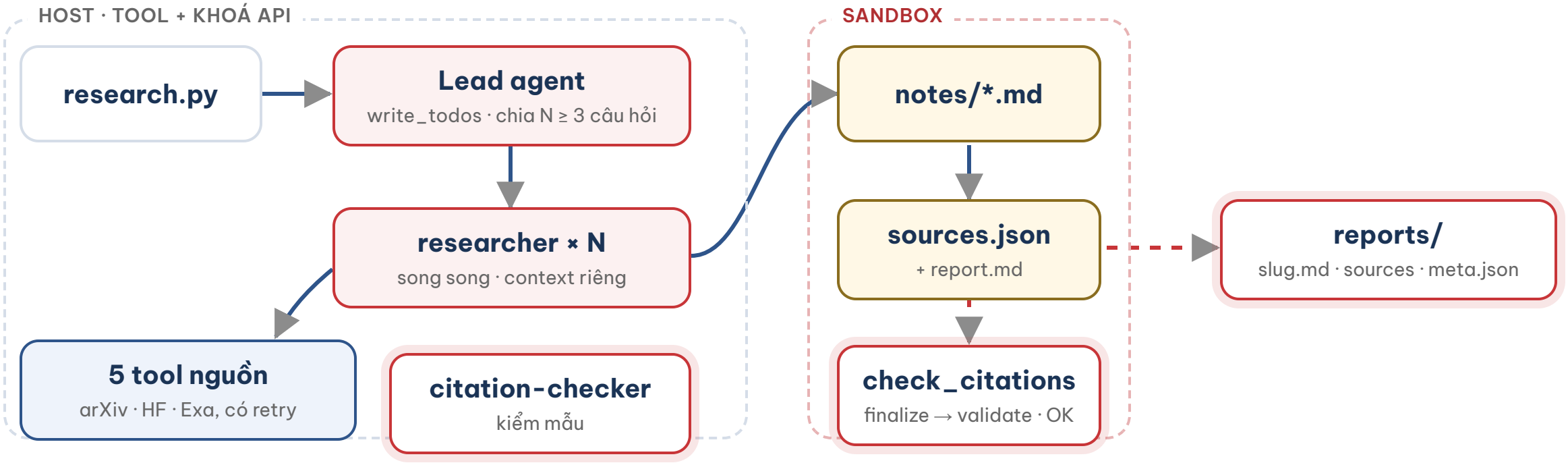
# Trợ lý khiếu nại: LangGraph shell, deep agent core, approval gate bền




Checkpoint, trace và đánh giá chạy dưới toàn bộ graph.

Nguồn: Kiến trúc của bài giảng, tổng hợp từ phần 01–06

# Lab: Deep Research Agent — khoá ở host, tệp trong sandbox




Finalizer rồi validator, bằng execute; tải về.

Nguồn: github.com/VinUni-AI20k/K4-Day23-AdvanceDeepAgents-Labs · README, GUIDE · deepagents 0.7.21 + Daytona

# Lab: bốn tệp theo thứ tự, năm chủ đề, chấm trên 100 điểm

1 **check_citations.py** — thuần Python, chạy trong sandbox.

2 **tools.py** — `with_retry` + 5 tool; thử `python tools.py`.

3 **agents.py** — prompt lead, researcher, 2 subagent.

4 **research.py** — sandbox, `meta.json`; 5 chủ đề, rồi `self_check.py`.

**CHẤM**

Tool 20 · Agent 20 · Sandbox 10 · Trích dẫn 15 · Báo cáo 25 · Repo 10.

**KHÔNG ĐƯỢC**

Sửa `model.py`, `sandbox.py`; sửa tay báo cáo; commit `.env`.

Nguồn: RUBRIC.md, GUIDE.md của repo lab · nộp: public repo có mã + reports/ của đủ 5 chủ đề

# Bốn cơ chế của buổi học mà lab bắt bạn tự viết

### Context isolation

Subagent chỉ thấy tin giao việc: gửi đủ chủ đề, câu hỏi, đường dẫn.

### Sandbox ≠ an toàn

Web là dữ liệu không tin cậy; khoá ở lại host.

### Retry đúng cách

Backoff + jitter + Retry-After. Exa báo rate limit bằng HTTP 200.

### Deterministic shell

Agent viết, code kiểm: chưa in OK thì chưa xong.

Nguồn: GUIDE.md §1.1, §1.4, §2, §4 của repo lab

# Trước khi ship agent: tám câu phải trả lời được

01 Ai chọn bước tiếp theo ở từng node?

02 Vòng lặp có budget và điều kiện dừng?

03 Hành động không đảo ngược có cổng?

04 Side effect có idempotency key?

05 Checkpointer và durability nào?

06 Luật nào còn nằm trong prompt?

07 Trace có chỉ ra bước gây lỗi?

08 Eval nào chặn bản mới trong CI?

# Agent evaluation ở ba lớp, với một cổng chặn trong CI

### Final output

Quyết định hoàn tiền có đúng chính sách? So với nhãn của người.

### Trajectory

Không hoàn tiền chưa duyệt, không trả trùng, ≤ 15 lượt?

### Regression set

Vài chục ca thật; vi phạm bất biến hay tụt điểm thì chặn merge.

**Cổng đặt trước khi ship, không phải sau sự cố.**

Nguồn: Chi tiết ở Day 14 (Evaluation) · docs.langchain.com/langsmith/evaluation

# Mang về từ cả buổi

01 **Ai chọn bước tiếp theo**: chọn theo bài toán.

02 LangGraph dựng deterministic shell, bền.

03 Deep Agents dựng agentic core.

04 Framework 2026 hội tụ: graph bền + harness.

05 Frontier: học điều phối, đóng băng agent thành workflow.

# Tài liệu tham khảo chính


- Anthropic, Building effective agents (12/2024); Multi-agent research system (06/2025); Effective context engineering (09/2025); Harness design for long-running apps (03/2026)
- LangChain: LangGraph & LangChain 1.0 (10/2025); Deep Agents (07/2025); Deep Agents v0.7 (07/2026); docs.langchain.com/oss/python
- OpenAI, A practical guide to building agents; Google Cloud, Choose a design pattern for agentic AI (05/2026)
- HumanLayer, 12-Factor Agents; Cognition, Don't Build Multi-Agents (06/2025), Multi-Agents: What's Actually Working (04/2026)
- Google ADK, Microsoft Agent Framework, OpenAI Agents SDK, Claude Agent SDK, Pi, DeepSeek Harness, Hermes Agent: repo và release chính thức (đọc 08/10/2026)
- Kim et al., Towards a Science of Scaling Agent Systems, arXiv 2512.08296
- Cemri et al., Why Do Multi-Agent LLM Systems Fail?, arXiv 2503.13657 · Who&When, 2505.00212
- AFlow 2410.10762 · MaAS 2502.04180 · Darwin Gödel Machine 2505.22954
- Conductor 2512.04388 · ToolOrchestra 2511.21689
- TraceCompiler 2608.02680 · Agent-C 2512.23738 · Lean4Agent 2606.06523 · Chronicle 2609.20625
- ACE 2510.04618 · Recursive LMs 2512.24601 · Chroma, Context Rot (07/2025) · METR time horizons (2026)


Nguồn: Danh sách đầy đủ có link và ngày: _research/L1–L5, S1–S2 trong thư mục slide

# Cảm ơn

Câu hỏi · slide, nghiên cứu và lab trên VLearn